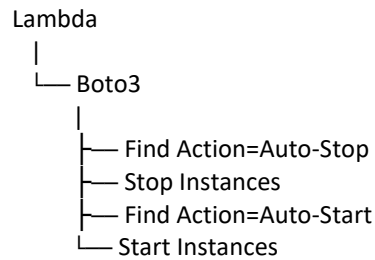


Assignment 1: Automated EC2 Start/Stop Using Lambda

Architecture



Step 1: Create EC2 Instances

Create two EC2 instances:

Instance 1

Tag:

Key: Action
Value: Auto-Stop

Instance 2

Tag:

Key: Action
Value: Auto-Start

Step 2: Create IAM Role

IAM → Roles → Create Role

Trusted Entity:

AWS Service
Lambda

Attach Policy:

AmazonEC2FullAccess

Role Name:

Lambda-EC2-Manager

Step 3: Create Lambda Function

Runtime:

Python 3.12

Execution Role:

Lambda-EC2-Manager

Step 4: Lambda Code

```
import boto3

ec2 = boto3.client('ec2')

def lambda_handler(event, context):

    stop_instances = []
    start_instances = []

    response = ec2.describe_instances(
        Filters=[
            {
                'Name': 'tag:Action',
                'Values': ['Auto-Stop']
            }
        ]
    )

    for reservation in response['Reservations']:
        for instance in reservation['Instances']:
            stop_instances.append(instance['InstanceId'])

    if stop_instances:
        ec2.stop_instances(InstanceIds=stop_instances)

    response = ec2.describe_instances(
```

```

Filters=[
    {
        'Name': 'tag:Action',
        'Values': ['Auto-Start']
    }
]
)

for reservation in response['Reservations']:
    for instance in reservation['Instances']:
        start_instances.append(instance['InstanceId'])

if start_instances:
    ec2.start_instances(InstanceIds=start_instances)

print("Stopped:", stop_instances)
print("Started:", start_instances)

return {
    'statusCode': 200
}

```

Expected Lambda Output

Stopped: ['i-0ab123456789']
 Started: ['i-0cd987654321']

Screenshots

The screenshot displays the AWS Management Console interface. On the left, the navigation menu shows various AWS services, with 'EC2' selected. The main content area is titled 'Instances (1/2)' and shows a table of EC2 instances. Two instances are listed: 'Server-AutoStart' (Instance ID: i-0684810877462126, Status: Stopped, Instance type: t3.micro) and 'Server-AutoStop' (Instance ID: i-04bc591a04a64c63d, Status: Stopped, Instance type: t3.micro). Below the table, the details for the 'Server-AutoStop' instance are shown. The 'Instance summary' section includes the Instance ID, IP name, Hostname type, Amazon private resource DNS name, Auto-assigned IP address, IAM role, and Instance profile. The 'Public IPv4 address' section shows the Public IP address, Instance state, Private IP DNS name, Instance type, VPC ID, Subnet ID, and Instance ARN. The 'Private IPv4 addresses' section shows the Private IP addresses, Public DNS, Elastic IP addresses, AWS Compute Optimizer finding, Auto Scaling Group name, and Managed state.

Lambda-EC2-Manager

Function overview

Diagram Template

Layers

Throttle Copy ARN Actions

Function ARN: arn:aws:lambda:us-east-1:352779209001:function:Lambda-EC2-Manager

Expert to Infrastructure Composer Download

Code source

```
1 import boto3
2
3 ec2 = boto3.client('ec2')
4
5 def lambda_handler(event, context):
6     stop_instances = []
7     start_instances = []
8
9     response = ec2.describe_instances(
10         filters=[
11             {
12                 'Name': 'tag:Action',
13                 'Values': ['Auto-Stop']
14             }
15         ]
16     )
17
18     for reservation in response['Reservations']:
19         for instance in reservation['Instances']:
20             stop_instances.append(instance['InstanceId'])
21
22     if stop_instances:
23         ec2.stop_instances(InstanceIds=stop_instances)
24
25     response = ec2.describe_instances(
26         filters=[
27             {
28                 'Name': 'tag:Action',
29                 'Values': ['Auto-Start']
30             }
31         ]
32     )
```

ENVIRONMENT VARIABLES

Choose a tutorial

Create a simple web app

Start tutorial

EC2

Instances (2)

Save filter rule Find instance by attribute or tag (aws-remote)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs	Monitoring	Security group name	Key name	Launch ID
Server-AutoStart	i-00084810977462126	Running	t3.micro	Initializing	us-east-1d	ec2-34-203-220-215.m...	34.203.220.215	-	-	disabled	launch-wizard-3	assignment	2026/04/...
Server-AutoStop	i-04bc5f91a04a0463d	Stopped	t3.micro	-	us-east-1d	-	-	-	-	disabled	launch-wizard-4	assignment	2026/04/...

Select an instance

Assignment 2: S3 Cleanup (Delete Files Older Than 30 Days)

Step 1: Create Bucket

Example:

aws-cleanup-demo-bucket

Upload:

file1.txt
file2.txt
file3.txt

Step 2: IAM Role

Attach:

AmazonS3FullAccess

Step 3: Lambda Code

```
import boto3
from datetime import datetime, timezone, timedelta

s3 = boto3.client('s3')

BUCKET_NAME = "aws-cleanup-demo-bucket"

def lambda_handler(event, context):

    cutoff = datetime.now(timezone.utc) - timedelta(days=30)

    response = s3.list_objects_v2(Bucket=BUCKET_NAME)

    for obj in response.get('Contents', []):

        if obj['LastModified'] < cutoff:

            s3.delete_object(
                Bucket=BUCKET_NAME,
                Key=obj['Key']
```

```
)

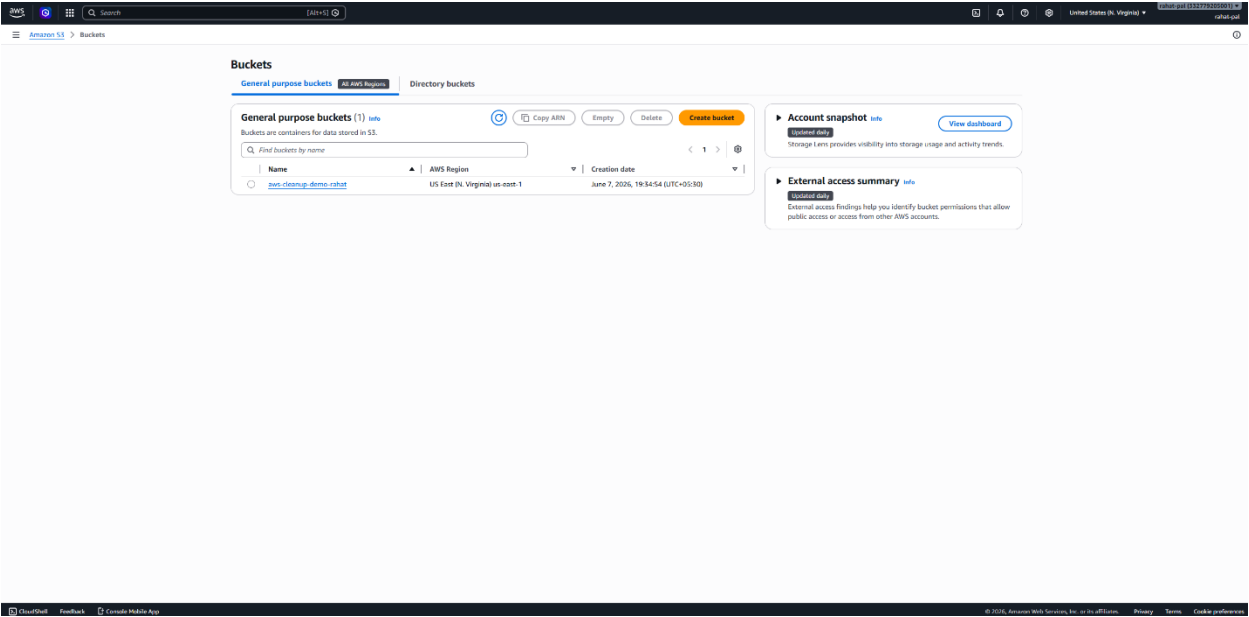
print(f"Deleted: {obj['Key']}")

return {
    'statusCode': 200
}
```

Expected Output

Deleted: old_backup.zip
Deleted: old_logs.txt

Screenshots



aws-cleanup-demo-rahaf info

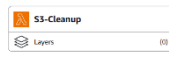
Objects Metadata Properties Permissions Metrics Management File systems - new Access Points

Objects (3)						
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 Inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more						
Find objects by prefix						
☐	Name	Type	Last modified	Size	Storage class	
☒	test3.txt	txt	June 7, 2026, 19:37:21 (UTC+05:30)	0 B	Standard	
☐	test3.txt	txt	June 7, 2026, 19:37:22 (UTC+05:30)	0 B	Standard	
☒	test3.jpg	jpg	June 7, 2026, 19:37:22 (UTC+05:30)	142.0 B	Standard	

S3-Cleanup

Function overview info

Diagram Template



+ Add trigger

+ Add destination

Function ARN
arn:aws-lambda:us-east-1:3327970301:func:S3-Cleanup
Description
-
Last modified
5 seconds ago

Export to Infrastructure Composer Download

Code Test Monitor Configuration Aliases Versions

Code source info

```
1 import boto3
2 from datetime import datetime, timedelta
3
4 s3 = boto3.client('s3')
5
6 BUCKET_NAME = "aws-cleanup-demo-rahaf"
7
8 def lambda_handler(event, context):
9     cutoff = datetime.now(timedelta.utcnow()) - timedelta(days=10)
10
11     response = s3.list_objects_v2(Bucket=BUCKET_NAME)
12
13     for obj in response.get('Contents', []):
14         if obj['LastModified'] < cutoff:
15
16             s3.delete_object(
17                 Bucket=BUCKET_NAME,
18                 Key=obj['Key']
19             )
20             print(f'Deleted: {obj["Key"]}')
21
22     return {
23         'statusCode': 200
24     }
```

Info Tutorials

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

Learn more

Start tutorial

DiagramTemplate

S3-Cleanup

Layers

0/0

+ Add trigger

+ Add destination

Function ARN

arn:aws:lambda:us-east-1:332779309001:function:S3-Cleanup

Description

Last modified

35 seconds ago

CodeTestMonitorConfigurationAliasesVersions

Executing function succeeded [logs ↗](#)

Details

```
{
  "statusCode": 200
}
```

Summary

Code SHA-256

8ba7c1a9a4a16f4e4g5f8E5kGE+3baeVQ25wWQKwWD=

Function version

SLATEST

Duration

501.57 ms

Resources configured

128 MB

Init duration

506.36 ms

Log output

The area below shows the last 4 KB of the execution log. [Click here ↗](#) to view the corresponding CloudWatch log group.

START RequestId: a8f32077-8142-468b-a77e-8cc279d5a6f6 Version: SLATEST

END RequestId: a8f32077-8142-468b-a77e-8cc279d5a6f6

REPORT RequestId: a8f32077-8142-468b-a77e-8cc279d5a6f6 Duration: 501.57 ms Billed Duration: 608 ms Memory Size: 128 MB Max Memory Used: 96 MB Init Duration: 506.36 ms

Execution time

11 seconds ago

Request ID

a8f32077-8142-468b-a77e-8cc279d5a6f6

Billed duration

608 ms

Max memory used

96 MB

Test event ↗

To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.

Test event action

Create new event

ⓘ Edit saved event

Delete

CloudWatch Logs Live Tail

Save

Test

InfoTutorials

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL, that outputs a webpage.
- Invoke your function through its function URL.

[Learn more ↗](#)

Start tutorial

CloudTrailFeedbackCreate Profile App

© 2020, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCreate preferences

Assignment 3: Detect Unencrypted S3 Buckets

IAM Role

Attach:

AmazonS3ReadOnlyAccess

Lambda Code

```
import boto3
```

```
from botocore.exceptions import ClientError
```

```
s3 = boto3.client('s3')
```

```
def lambda_handler(event, context):
```

```
    encrypted_buckets = []
```

```
    unencrypted_buckets = []
```

```
    buckets = s3.list_buckets()
```

```
    for bucket in buckets['Buckets']:
```

```
        bucket_name = bucket['Name']
```

```
        try:
```

```
            s3.get_bucket_encryption(Bucket=bucket_name)
```

```
            encrypted_buckets.append(bucket_name)
```

```
except ClientError as e:

    error_code = e.response['Error']['Code']

    if error_code == 'ServerSideEncryptionConfigurationNotFoundError':

        unencrypted_buckets.append(bucket_name)

print("=== Encryption Report ===")

if unencrypted_buckets:

    print("\nUnencrypted Buckets:")

    for bucket in unencrypted_buckets:

        print(bucket)

else:

    print("\nNo unencrypted buckets found.")

print("\nEncrypted Buckets:")

for bucket in encrypted_buckets:

    print(f"{bucket} - Encrypted")

return {

    "statusCode": 200,

    "message": "All buckets checked successfully",

    "unencrypted_buckets": unencrypted_buckets,
```

```
"encrypted_buckets": encrypted_buckets

}

Expected Output:

{

  "statusCode": 200,

  "message": "All buckets checked successfully",

  "unencrypted_buckets": [],

  "encrypted_buckets": [

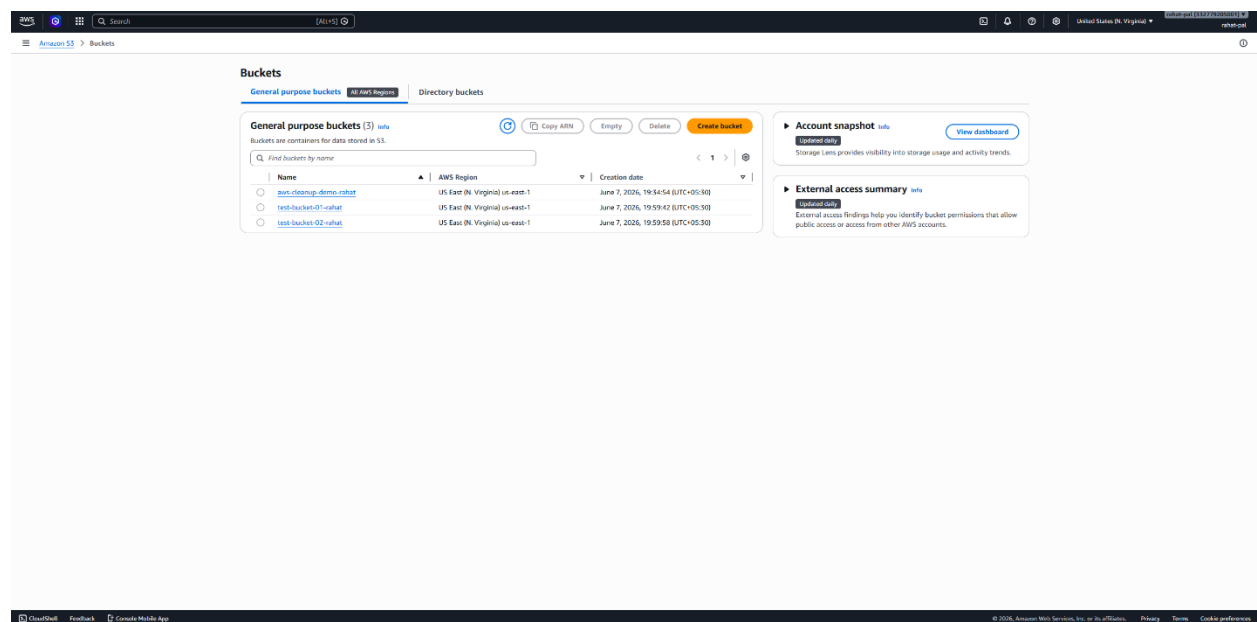
    "aws-cleanup-demo-rahath",

    "test-bucket-01-rahath",

    "test-bucket-02-rahath"

  ]

}
```



Introducing the AWS Serverless generative AI webinar series

Join AWS Serverless experts for a hands-on workshop on Agentic AI, Intelligent Document Processing, and building real-time APIs for chatbots.

See more details

Functions (3)

Search by attributes or search by keyword

Function name	Description	Package type	Runtime	Type	Last modified
S3 Cleanup	-	Zip	Python 3.12	Standard	18 minutes ago
Lambda-EC2-Manager	-	Zip	Python 3.12	Standard	52 minutes ago
DetectUnencryptedS3Buckets	-	Zip	Python 3.12	Standard	18 seconds ago

Info

Tutorials

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage.
- Invoke your function through its function URL.

Learn more

Start tutorial

CloudShell

Feedback

Download mobile app

© 2021, Amazon Web Services, Inc. or its affiliates

Privacy

Terms

Create preferences

Successfully updated the function "DetectUnencryptedS3Buckets."

Code

Test

Monitor

Configuration

Aliases

Versions

Code source

Info

Open in Visual Studio Code

Update

lambda_function.py

```
1 import boto3
2 from botocore.exceptions import ClientError
3
4 s3 = boto3.client('s3')
5
6 def lambda_handler(event, context):
7     unencrypted_buckets = []
8     unencrypted_buckets = []
9
10    buckets = s3.list_buckets()
11
12    for bucket in buckets['Buckets']:
13        bucket_name = bucket['Name']
14
15        try:
16            s3.get_bucket_encryption(Bucket=bucket_name)
17            encrypted_buckets.append(bucket_name)
18
19        except ClientError as e:
20            error_code = e.response['Error']['Code']
21            if error_code == "ServerSideEncryptionConfigurationNotFoundError":
22                unencrypted_buckets.append(bucket_name)
23
24    print(f"Encryption Report")
25
26    if unencrypted_buckets:
27        print(f"Unencrypted Buckets:")
28        for bucket in unencrypted_buckets:
29            print(bucket)
```

Code properties

Info

Package size

574 bytes

SHA256 hash

646c04b9b9c0c0f6a2b4c0f649c2f0d9b9a4b0a

Last modified

1 minute ago

Runtime settings

Info

Edit

Edit runtime management configuration

Info

Tutorials

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage.
- Invoke your function through its function URL.

Learn more

Start tutorial

CloudShell

Feedback

Download mobile app

© 2021, Amazon Web Services, Inc. or its affiliates

Privacy

Terms

Create preferences

👉 Successfully updated the function "DetectUnencryptedS3Buckets".

[Code](#) | [Test](#) | [Monitor](#) | [Configuration](#) | [Aliases](#) | [Versions](#)

✔ Executing function: succeeded ([logs L2](#))

```
{
  "statusCode": 200,
  "message": "All buckets checked successfully",
  "unencrypted_buckets": [],
  "encrypted_buckets": [
    "aws-elasticmapreduce-elasticmapreduce",
    "test-bucket-01-elasticmapreduce",
    "test-bucket-02-elasticmapreduce"
  ]
}
```

Summary

Code SHA-256
6AdvQqBHqPoUGAFshZv4vCrFz4jXcZFDCBHerU+9e

Execution time
2 seconds ago

Function version
\$LATEST

Request ID
ff64dff3-d1ca-4562-9791-9c065af7d1d3

Duration
929.22 ms

Billed duration
1489 ms

Resources configured
128 MB

Max memory used
95 MB

init duration
558.99 ms

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

[illegible]Test event [info](#)

To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.

Delete

CloudWatch Logs Live Tail

Save

Test

CloudShell Feedback Console Mobile App

Assignment 4: EBS Snapshot Automation

Step 1: Get Volume ID

Example:

vol-0abcd123456789

Step 2: IAM Role

Attach:

AmazonEC2FullAccess

Step 3: Lambda Code

```
import boto3
from datetime import datetime, timezone, timedelta

ec2 = boto3.client('ec2')

VOLUME_ID = "vol-0abcd123456789"

def lambda_handler(event, context):

    snapshot = ec2.create_snapshot(
        VolumeId=VOLUME_ID,
        Description="Automated Backup"
    )

    print("Created Snapshot:",
          snapshot['SnapshotId'])

    cutoff = datetime.now(timezone.utc) - timedelta(days=30)

    snapshots = ec2.describe_snapshots(
        OwnerIds=['self']
    )

    for snap in snapshots['Snapshots']:

        if snap['StartTime'] < cutoff:

            ec2.delete_snapshot(
                SnapshotId=snap['SnapshotId']
```

```
)

print(
    "Deleted:",
    snap['SnapshotId']
)

return {
    'statusCode': 200
}
```

Expected Output

Created Snapshot: snap-01ab234cd567

Deleted: snap-09xy123zz456

Deleted: snap-07gh987jk654

Screenshot

The screenshot shows the AWS Management Console interface for the EC2 service. The main content area displays a table of instances. The table has columns for Name, Instance ID, Instance state, Instance type, Status check, Availability Zone, Public IPv4 DNS, Public IPv4 address, Elastic IP, IPv6 IPs, Monitoring, Security group name, Key name, and Launch time. Two instances are listed: 'Server-AutoStart' (ID: i-09684830977462126) and 'aws-snapshot-demo' (ID: i-09908120087938627). Both are in the 'Running' state. The left sidebar contains a navigation menu with categories like EC2, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The top navigation bar includes the AWS logo, search bar, and account information.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 address	Elastic IP	IPv6 IPs	Monitoring	Security group name	Key name	Launch time
Server-AutoStart	i-09684830977462126	Running	t1.micro	1/1 checks passed	us-east-1d	ec2-34-205-220-215.eu...	34.205.220.215	-	-	disabled	launch-wizard-3	assignment	2016/06/...
aws-snapshot-demo	i-09908120087938627	Running	t1.micro	1/1 checks passed	us-east-1d	ec2-50-19-139-129.eu...	50.19.139.129	-	-	disabled	launch-wizard-5	assignment	2016/06/...

aws

Search

[Alt+S]

United States (N. Virginia)

rahul-pal (532779205001)

rahul-pal

Lambda > Functions

Introducing the AWS Serverless generative AI webinar series
Join AWS Serverless experts for a hands-on workshops on Agentic AI, Intelligent Document Processing, and building real-time APIs for chatbots.
[See more details](#)

Functions (4)
Search by attributes or search by keyword

Last fetched 6/7/2026, 8:43:34 PM
Actions
Create function

	Function name	Description	Package type	Runtime	Type	Last modified
☐	DetectUnencryptedS3Buckets	-	Zip	Python 3.12	Standard	31 minutes ago
☐	S3-Cleanup	-	Zip	Python 3.12	Standard	59 minutes ago
☐	Lambda-EC2-Manager	-	Zip	Python 3.12	Standard	2 hours ago
☐	EBSSnapshotAutomation	-	Zip	Python 3.12	Standard	8 minutes ago

Info

Tutorials

Choose a tutorial
Learn how to implement common use cases in AWS Lambda.

Create a simple web app
In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)
[Start tutorial](#)

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

aws

Search

[Alt+S]

United States (N. Virginia)

rahul-pal

Lambda > Functions > EBSSnapshotAutomation

Throttle

Copy ARN

Actions

Export to Infrastructure Composer

Download

Function overview

Diagram

Template

EBSSnapshotAutomation

Layers

0

+ Add trigger

+ Add destination

Function ARN
arn:aws:lambda:us-east-1:532779205001:function:EBSSnapshotAutomation

Description
-

Last modified
8 minutes ago

Code

Test

Monitor

Configuration

Aliases

Versions

Code source

Open in Visual Studio Code

Update

env:code

EBSSNAPSHOTAUTOMATION

lambda_function.py

def lambda_handler(event, context):
7
8 volumes = ec2.describe_volumes()["Volumes"]
9
10 snapshots_created = []
11
12 for volume in volumes:
13 volume_id = volume["VolumeId"]
14
15 response = ec2.create_snapshot(
16 VolumeId=volume_id,
17 Description=f"Automated snapshot of {volume_id}"
18)
19
20 snapshots_created.append(
21 {
22 "VolumeId": volume_id,
23 "SnapshotId": response["SnapshotId"]
24 })
25
26 return {
27 "statusCode": 200,
28 "SnapshotsCreated": snapshots_created
29 }

TEST EVENTS (NONE SELECTED)
+ Create new test event

ENVIRONMENT VARIABLES

Info

Tutorials

Choose a tutorial
Learn how to implement common use cases in AWS Lambda.

Create a simple web app
In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)
[Start tutorial](#)

Executing Function: succeeded [Logs](#)

Details

```
{
  "statusCode": 200,
  "headers": {
    "Content-Type": "application/json"
  },
  "body": {
    "message": "Function executed successfully"
  }
}
```

Summary

Code SHA-256	Execution time
jkavZKwD/BAJ6dJSE4DZB0Ugk7v9n7eBb7ZEds	1 minute ago
Function version	Request ID
SLATEST	44a0135-6ee-4590-bc3f-5a05f783ba3
Duration	Billed duration
1450.73 ms	1451 ms
Resources configured	Max memory used
128 MB	101 MB

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

START RequestID: 44a0135-6ee-4590-bc3f-5a05f783ba3 Version: SLATEST
END RequestID: 44a0135-6ee-4590-bc3f-5a05f783ba3
REPORT RequestID: 44a0135-6ee-4590-bc3f-5a05f783ba3 Duration: 1450.73 ms Billed Duration: 1451 ms Memory Size: 128 MB Max Memory Used: 101 MB

Test event Info

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action

☒ Create new event ☐ Edit saved event

Invocation type

☒ Synchronous
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous
Invokes the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like S3, SNS, or Eventbridge.

Choose a tutorial

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

[Start tutorial](#)

Snapshots (5) Info

Last updated 3 minutes ago [Recycle Bin](#) [Actions](#) [Create snapshot](#)

Snapshot scope: Owned by me

☐	Name	Snapshot ID	Full snapshot size	Volume size	Description	Storage tier	Snapshot status	Started	Progress
☐		snap-0bdee62908cfe6b35	1.62 GiB	8 GiB	Automated snapshot of vol...	Standard	Completed	2026/06/07 20:35 GMT+5...	100%
☐		snap-0a0ff4daa413d57f2	1.62 GiB	8 GiB	Automated snapshot of vol...	Standard	Completed	2026/06/07 20:36 GMT+5...	100%
☐	Link	snap-0a1e471a4c8b5c9ea	1.61 GiB	8 GiB	Automated snapshot of vol...	Standard	Completed	2026/06/07 20:35 GMT+5...	100%
☐		snap-05df79d19665bb021	1.61 GiB	8 GiB	Automated snapshot of vol...	Standard	Completed	2026/06/07 20:36 GMT+5...	100%
☐		snap-0dc66ce0a9527d05	1.61 GiB	8 GiB	Automated snapshot of vol...	Standard	Completed	2026/06/07 20:35 GMT+5...	100%
☐		snap-006f9580df694f817	1.61 GiB	8 GiB	Automated snapshot of vol...	Standard	Completed	2026/06/07 20:36 GMT+5...	100%

Select a snapshot above.